Competitive Programming: Algorithm Reference & Roadmap

Target: 2400+ Codeforces

Note: Items marked **[ADDED]** in blue were not in the original document and have been supplemented based on coverage analysis.

How to Use This Document

Two parts:

Part 1 — Reference: What to learn, organized by tier and topic family, with depth labels

Part 2 — Roadmap: How to learn it, in what order, to what depth, and when to advance

Depth Labels

Every item is tagged:

- [Implement] — Must code from scratch in contest conditions without hesitation
- [Template] — Understand fully; use a pre-written and tested template in contests
- [Know] — Understand how it works and when to apply it; don't need to code it in contest

PART 1: REFERENCE

TIER 1 — Absolute Foundations

CF ~800–1200. Must know before everything else.

Data Structures

- Array / dynamic array (vector) [Implement]
- String and string operations [Implement]
- Stack [Implement]
- Queue [Implement]
- Deque [Implement]
- Hash map (unordered_map) [Implement]
- Hash set (unordered_set) [Implement]
- Ordered map (std::map) [Implement]
- Ordered set, multiset (std::set) [Implement]
- Priority queue — max-heap and min-heap [Implement]
- Pair, tuple [Implement]

Algorithms & Techniques

- Sorting — std::sort, custom comparator [Implement]
- Binary search — lower_bound, upper_bound, manual [Implement]
- Two pointers [Implement]
- Sliding window — fixed and variable size [Implement]
- Prefix sums (1D) [Implement]
- Difference arrays [Implement]
- Brute force / exhaustive enumeration [Implement]
- Recursion [Implement]
- Backtracking [Implement]

- Greedy (basic — sort by key, interval scheduling) [Implement]
- Divide and conquer (basic) [Implement]

Math

- Modular arithmetic — add, subtract, multiply, overflow handling [Implement]
- GCD / LCM — Euclidean algorithm [Implement]
- Fast exponentiation (binary exponentiation) [Implement]
- Sieve of Eratosthenes [Implement]
- Trial division / prime factorization [Implement]

C++ Patterns

- Fast I/O — ios_base::sync_with_stdio(false), cin.tie(NULL) [Implement]
- auto and range-based for loops [Implement]
- Lambda functions [Implement]
- STL algorithms: sort, reverse, unique, lower_bound, upper_bound, max_element, min_element, accumulate, fill, count, find [Implement]
- __builtin_popcount, __builtin_clz, __builtin_ctz [Implement]
- Bitwise operators and basic bit manipulation [Implement]
- Stress testing setup — brute force + random input generator + diff checker [Implement]

Note on stress testing: This is missing from most lists but is one of the highest-value skills. Build it once in Phase 0 and use it in every phase. It will save more time than almost any algorithm.

TIER 2 — Beginner-Intermediate

CF ~1200–1700. Div. 3 / Div. 2 A–B.

Data Structures

- DSU / Union-Find — union by rank, path compression [Implement]
- Monotonic stack [Implement]
- Monotonic deque [Implement]

Graph Algorithms

- BFS — unweighted shortest path, level graph [Implement]
- DFS — traversal, timestamps, tree/non-tree edge classification [Implement]
- Connected components [Implement]
- Flood fill [Implement]
- Bipartite check [Implement]
- Cycle detection — directed and undirected [Implement]
- Topological sort — DFS-based [Implement]
- Topological sort — Kahn's algorithm (BFS-based) [Implement]

Dynamic Programming

- Linear DP [Implement]
- 0/1 Knapsack [Implement]
- Unbounded knapsack [Implement]
- Bounded knapsack [Implement]
- Coin change — min coins and count ways [Implement]
- Longest Increasing Subsequence — $O(n^2)$ [Implement]
- Longest Common Subsequence [Implement]
- Grid DP [Implement]
- DP with prefix sum optimization [Implement]

Math

- Extended Euclidean algorithm [Implement]
- Modular inverse — Fermat's little theorem [Implement]
- Modular inverse — extended Euclidean [Implement]
- Combinatorics: nCr, Pascal's triangle, factorials with mod [Implement]
- Inclusion-exclusion principle (basic) [Implement]
- Integer overflow awareness — long long, unsigned long long, __int128 [Implement]
- Number of divisors / sum of divisors [Implement]

Strings

- Brute force string matching [Implement]
- Palindrome checking [Implement]
- Basic polynomial hashing [Implement]

C++ Patterns

- Coordinate compression [Implement]
- 1-indexed vs 0-indexed conversion awareness [Implement]
- Printing precision for floating point [Implement]
- Custom structs with comparators [Implement]
- **[ADDED] Interactive problem handling — flushing output (cout << flush / endl), binary search interaction, adaptive query strategies [Implement]**

TIER 3 — Intermediate

CF ~1700–2100. Div. 2 C–E / USACO Silver.

Data Structures

- Segment tree — point update, range query [Implement]
- Fenwick tree / BIT — 1D, point update, range sum [Implement]
- Fenwick tree — 2D [Implement]
- Sparse table — static RMQ, O(1) query [Implement]
- Trie — string trie and binary trie [Implement]

Graph Algorithms

- Dijkstra's algorithm with priority queue [Implement]
- Bellman-Ford [Implement]
- SPFA [Implement]
- Floyd-Warshall [Implement]
- Kruskal's MST [Implement]
- Prim's MST [Implement]
- 0-1 BFS (deque-based for 0/1 edge weights) [Implement]
- Multi-source BFS [Implement]
- Functional graph traversal — each node out-degree = 1 [Implement]
- Grid graphs / implicit graphs [Implement]

Dynamic Programming

- DP on trees (basic — subtree DP) [Implement]
- Interval DP [Implement]
- Bitmask DP — subset enumeration [Implement]
- Digit DP [Implement]
- Matrix exponentiation for linear recurrences [Implement]
- DP with segment tree / BIT optimization [Implement]
- SOS DP (Sum over Subsets) [Implement]

- **[ADDED] Expected value DP — linearity of expectation, expected number of steps, basic probability transitions over states [Implement]**

Math

- Chinese Remainder Theorem [Implement]
- Euler's totient function φ(n) [Implement]
- Linear sieve — primes and multiplicative functions in O(n) [Implement]
- Möbius function μ(n) [Implement]
- Multiplicative functions [Know]
- Miller-Rabin primality test [Implement]
- Pollard's rho factorization [Template]
- Precomputing factorials and inverse factorials [Implement]

Strings

- KMP — failure function [Implement]
- Z-algorithm [Implement]
- Rabin-Karp / polynomial rolling hash [Implement]
- Double hashing — anti-collision [Implement]
- Manacher's algorithm (palindromes in O(n)) [Implement]

Geometry (Basic)

- Cross product, dot product [Implement]
- Orientation test [Implement]
- Line intersection check [Implement]
- Distance from point to segment [Implement]

C++ Patterns

- Binary search on the answer (parametric search) [Implement]
- Offline processing — sorting queries by parameter [Implement]
- Event-based sweep line [Implement]
- Meet in the middle — split and enumerate halves [Implement]
- Small-to-large merging (DSU / sets) [Implement]
- Mo's algorithm — O((n+q)√n) offline range queries [Implement]

TIER 4 — Advanced

CF ~2100–2600. Div. 1 C–D / USACO Gold–Platinum. This is where 2400 is earned.

Data Structures

- Segment tree with lazy propagation — range update, range query [Implement]
- Segment tree merging [Implement]
- Persistent segment tree [Template]
- Li Chao tree [Implement]
- Merge sort tree — offline range kth, range count [Template]
- Wavelet tree [Template]
- Ordered statistics tree — __gnu_pbds::tree, order_of_key, find_by_order [Implement]
- Mo's algorithm with rollback (updates) [Template]
- Mo's algorithm on trees [Template]
- Sqrt decomposition — block queries and updates [Implement]
- Implicit treap — split/merge operations [Template]

Graph Algorithms

- Tarjan's SCC (strongly connected components) [Implement]

- Kosaraju's SCC [Implement]
- Bridges and articulation points — Tarjan [Implement]
- Biconnected components [Implement]
- Block-cut tree [Template]
- Bridge tree [Template]
- Euler path / circuit — Hierholzer's algorithm [Implement]
- Bipartite matching — Hopcroft-Karp [Implement]
- Max flow — Dinic's algorithm [Implement]
- Max flow — Ford-Fulkerson, Edmonds-Karp [Know]
- 2-SAT with implication graph + SCC [Implement]
- Min-cost max-flow (MCMF) [Template]
- Virtual / auxiliary nodes in graphs [Implement]
- Bipartite graph properties — König's theorem [Know]
- **[ADDED] Directed MST — Chu-Liu/Edmonds algorithm [Template]**

Tree Algorithms

- Binary lifting — jump pointers [Implement]
- LCA via binary lifting [Implement]
- LCA via Euler tour + RMQ [Implement]
- Offline LCA — Tarjan's offline algorithm [Know]
- Heavy-Light Decomposition + segment tree [Implement]
- Centroid decomposition [Implement]
- Euler tour / DFS in-out time flattening [Implement]
- DSU on tree (small-to-large) [Implement]
- Rerooting technique — tree DP from all roots [Implement]

Dynamic Programming

- Convex hull trick — linear (monotone) [Implement]
- Convex hull trick — with binary search [Implement]
- Divide and conquer DP optimization [Implement]
- Knuth's optimization (quadrangle inequality) [Implement]
- Aliens trick / WQS binary search [Template]
- Slope trick [Implement]
- DP on DAGs [Implement]
- DP + Aho-Corasick automaton [Template]
- Broken profile / profile DP [Template]
- **[ADDED] Markov chain DP — probability transitions over states, absorbing states, expected cost to absorption [Implement]**

Math

- FFT — polynomial multiplication [Implement]
- NTT — FFT mod prime [Implement]
- XOR basis / linear basis over GF(2) [Implement]
- Gaussian elimination over reals and GF(2) [Implement]
- Burnside's lemma [Know]
- Lagrange interpolation [Implement]
- Sprague-Grundy theorem [Implement]
- Nim and combinatorial game theory — Nim values, XOR strategy [Implement]
- Berlekamp-Massey — find minimal linear recurrence [Template]
- Lucas' theorem — nCr mod prime [Implement]
- Lifting the exponent lemma (LTE) [Know]

Strings

- Aho-Corasick automaton [Implement]
- Suffix array + LCP array — prefix doubling / SA-IS [Implement]

- Suffix automaton (SAM) [Template]
- Palindromic tree / Eertree [Template]
- Lyndon factorization / Duval's algorithm [Know]
- Booth's algorithm — lexicographically smallest rotation [Know]

Geometry (Intermediate)

- Convex hull — Andrew's monotone chain [Implement]
- Line segment intersection (general) [Implement]
- Point in polygon — ray casting, winding number [Implement]
- Rotating calipers [Template]
- Closest pair of points — divide and conquer [Implement]
- Shoelace formula — area of polygon [Implement]
- Minkowski sum [Know]

C++ Patterns

- Parallel binary search [Implement]
- Offline + persistent segment tree [Template]
- Rollback DSU — DSU with undo for offline dynamic connectivity [Implement]
- Bitset DP optimization — $O(n^2/64)$ [Implement]
- Random hashing with custom hash maps — anti-hack [Implement]
- #pragma GCC optimize("O3,unroll-loops") [Know]
- #pragma GCC target("avx2,bmi,bmi2,popcnt") [Know]
- HLD + lazy segment tree combinations [Implement]
- Centroid decomposition + auxiliary data structures [Template]

TIER 5 — Selective (2400 boundary only)

Learn these only after Tier 4 is fully solid. Only the items that realistically appear at CF 2400.

Worth Learning for 2400

- Segment tree beats (Ji driver segmentation — interval chmin/chmax) [Template]
- Virtual tree — auxiliary tree for a subset of nodes [Template]
- Fast Walsh-Hadamard Transform (FWT) — AND/OR/XOR convolutions [Implement]
- Subset sum convolution [Template]
- Formal power series — inverse, sqrt, exp, log [Template]
- DP on suffix automaton [Template]
- Suffix automaton with complex DP [Template]
- Half-plane intersection [Template]
- Primitive roots [Know]
- Discrete logarithm — Baby-step giant-step (BSGS) [Implement]
- Quadratic residues / Tonelli-Shanks — modular square root [Template]
- Dirichlet convolution [Know]
- Link-cut tree (LCT) [Template] — **[ADDED: depth upgraded from [Know] to [Template]. The original document labels it [Know] but also calls it a 'hard blocker when it does appear'. These are contradictory — [Know] is insufficient when a correct implementation is required to score the problem.]**

Skip Until 2800+

Everything else in the original Tier 5: matroid intersection, blossom algorithm, Voronoi/Delaunay, external memory, retroactive data structures, Gomory-Hu tree, kinetic heaps, Van Emde Boas, etc. These are essentially never the intended solution at 2400.

Cross-Cutting Paradigms

Not algorithms — these are problem-solving frameworks. Learn alongside Tier 2 and keep reinforcing throughout. These matter more than almost any individual algorithm at high ratings.

- Reduction — transform the problem into a known type before trying to solve it directly
- Contribution technique — compute each element's contribution across all subsets/pairs/intervals instead of iterating pairs
- Complementary counting — count the complement, subtract from total
- Observation-based invariants — identify what doesn't change; often unlocks the entire solution
- Construction problems — build a valid answer directly; prove it works rather than searching for it
- Parametric search — binary search on the answer, reduce to a feasibility check
- Graph modeling — recognizing when and how to build auxiliary graphs, virtual nodes, or edge transformations
- Offline vs online trade-offs — deliberately choosing to process all queries together
- Sweep line and event-based processing — process changes rather than states
- Divide and conquer on data — split the problem domain, not just the recursion
- Amortized analysis — recognizing that expensive operations are infrequent and budgeting accordingly
- Sqrt decomposition philosophy — balance between precompute cost and query cost; applies beyond the literal algorithm
- Lazy propagation philosophy — defer work until it is actually needed
- Randomization — when and why probabilistic approaches are correct and practical

PART 2: ROADMAP

Core Learning Methodology

The Principle

One concept → implement immediately → solve 5–10 problems → move on.

Never read everything in a tier first and then practice. You forget theory you didn't immediately apply. Never grind random problems without topic focus. You build no pattern recognition.

Depth Per Tier

Tier	What "knowing" means in practice
Tier 1	Code from memory in under 3 minutes
Tier 2	Code from memory in under 5 minutes, all edge cases handled
Tier 3	Implement from scratch in contest; 10–20 min for harder ones
Tier 4	Implement core from scratch; use clean template for complex variants
Tier 5	Recognize the technique; correctly adapt a template

Sequence Per Topic

Follow this exact sequence for every new topic:

1. Read or watch the concept — 20–40 minutes. One good source is enough. Understand the why, not just the what. CP-algorithms.com is the default reference.

2. Implement from scratch — Close the tutorial. Write the algorithm from memory. Get it compiling and passing a base case. No copy-paste. This is the step most people skip and most people regret.
3. Solve 3–5 focused problems — Problems where this technique is the clear intended solution. CSES and Codeforces EDU are best for this.
4. Solve 2–3 mixed problems — Problems where you must recognize the technique applies, not just apply it mechanically.
5. Write a clean template — One clean, tested, commented version. This is what you use in contests.

When You Are Stuck on a Problem

- Under 30 minutes: keep trying.
- 30–60 minutes: allow yourself one hint — just the technique name or the key observation, not the full solution.
- Over 60 minutes: read the full editorial. But implement the solution yourself after reading. Do not look at accepted code unless your own implementation fails after multiple attempts.

Contest Practice

Start from Phase 2 onwards:

- Do one virtual contest per week minimum. Time pressure is not optional to simulate — it is the skill.
- After every contest, virtual or rated: upsolve every problem you could not solve during the contest.
- Upsolving is where most of the learning happens. Do not skip it because you "understand the technique now." You must implement.

Error Log

Keep a running document with:

- The mistake type (wrong algorithm, overflow, off-by-one, wrong complexity estimate, misread problem)
- The problem link
- What you missed and why

Review before contests. Most people repeat the same 4–5 mistake types for months.

Phase 0 — Environment (1–2 weeks)

Goal: Correct setup and basic fluency before any algorithms.

What to do:

1. Set up your template: fast I/O, common typedefs, macros you actually use.
2. Build a stress tester. A brute force, a random input generator, and a diff checker script. This is worth more than almost any algorithm you will learn. Build it once; use it in every phase.
3. Solve 50–80 problems in CF 800–1100 range, just for fluency. If you have done this already, skip.

Done when: CF 800–1100 problems take under 10 minutes consistently.

Phase 1 — Tier 1 Mastery (6–10 weeks)

Goal: Every Tier 1 item is automatic. No thinking, no lookups.

Order:

1. C++ patterns and full STL fluency (Week 1) — all containers, all STL algorithms, bit manipulation, _builtin* functions
2. Sorting, binary search, two pointers, sliding window (Week 1–2)
3. Prefix sums and difference arrays (Week 2)
4. Recursion and backtracking (Week 2–3)
5. Greedy fundamentals — sorting by key, exchange arguments (Week 3–4)
6. Divide and conquer basics (Week 4)
7. Math foundations — modular arithmetic, GCD/LCM, fast exponentiation, sieve, trial division (Week 4–6)

Practice: CF 1000–1400. CSES Introductory Problems and Sorting/Searching sections.

Done when: CF 1200–1400 problems solve in under 15 minutes. You rarely TLE or RE on Tier 1 content. Binary search, two pointers, prefix sums are reflexive — you reach for them before thinking.

Phase 2 — Tier 2 Mastery (8–12 weeks)

Goal: Core graph traversal, basic DP, and basic math are solid. Comfortable in Div. 3.

Order:

1. Graph representation — adjacency list, edge list, when to use which (Week 1)
2. BFS fully — unweighted shortest path, connected components, flood fill, level graph (Week 1)
3. DFS fully — traversal, timestamps, tree vs back vs cross edges (Week 1–2)
4. DSU with union by rank and path compression (Week 2)
5. Bipartite check and cycle detection (Week 2)
6. Topological sort — both DFS-based and Kahn's (Week 3)
7. Monotonic stack (Week 3)
8. Monotonic deque (Week 4)
9. Linear DP — 1D and 2D (Week 4–5)
10. Knapsack variants — 0/1, unbounded, bounded (Week 5)
11. Classic DP — LIS $O(n^2)$, LCS, coin change (Week 6)
12. Grid DP (Week 6–7)
13. Math — extended GCD, modular inverse (both methods), combinatorics, inclusion-exclusion (Week 7–8)
14. Basic strings — hashing, palindrome, brute force matching (Week 8–9)
15. C++ patterns — coordinate compression, custom comparators, __int128 (Week 9)
16. **[ADDED] Interactive problems — output flushing, binary search interaction, adaptive strategies (Week 9–10)**

Practice: CF 1400–1800. CSES Graph Algorithms (basic) and Dynamic Programming sections. Start virtual Div. 3 contests weekly.

Done when: Div. 3 contests are comfortable — you finish them. CF 1600–1800 problems solve in under 30 minutes most of the time.

Phase 3 — Tier 3 Mastery (12–16 weeks)

Goal: Full intermediate toolkit. Segment tree, Dijkstra, advanced DP, string algorithms, parametric search. Target: ~2100.

Order:

1. Segment tree — point update, range query (Week 1–2). Do not rush this. Implement multiple variants before moving on. This is the most foundational data structure at this tier.
2. Fenwick tree / BIT (Week 2). Implement alongside segment tree. Understand when each is better.
3. Sparse table and static RMQ (Week 3)

4. Trie — string trie, then binary trie (Week 3–4)
5. Dijkstra, Bellman-Ford, SPFA, Floyd-Warshall (Week 4–5)
6. MST — Kruskal's and Prim's (Week 5)
7. 0-1 BFS, multi-source BFS, functional graphs (Week 5–6)
8. DP on trees — basic subtree DP (Week 6–7)
9. Interval DP (Week 7)
10. Bitmask DP (Week 7–8)
11. Digit DP (Week 8–9)
12. **[ADDED] Expected value DP — linearity of expectation, expected steps, basic probability transitions (Week 9)**
13. Matrix exponentiation (Week 9–10)
14. SOS DP — Sum over Subsets (Week 10)
15. DP with segment tree / BIT optimization (Week 10–11)
16. Math — CRT, totient, linear sieve, Miller-Rabin, Möbius, precomputed factorials/inverses (Week 11–12)
17. String algorithms — KMP, Z-algorithm, Rabin-Karp, double hashing, Manacher's (Week 12–14)
18. Basic geometry — cross/dot product, orientation test, line intersection (Week 14–15)
19. C++ patterns — binary search on answer, offline processing, sweep line, meet in the middle, Mo's algorithm, small-to-large merging (Week 15–16)

Practice: CF 1800–2200. Full CSES problem set pass. Virtual Div. 2 contests weekly.

Done when: You consistently solve CF 2000–2100 problems. You upsolve Div. 2 D problems. Segment tree and Dijkstra are not your bottleneck — your bottleneck is now problem-specific observation.

Watch out: Small-to-large merging and Mo's algorithm appear simple but consistently trip people. Spend extra time on them. They appear often in disguise.

Phase 4 — Tier 4 Mastery (20–30 weeks)

This is where 2400 is earned. Expect this phase to take longer than all previous phases combined.

Goal: Deep mastery of advanced data structures, tree algorithms, advanced DP, flows/matching, FFT, and core string algorithms.

Split into four blocks. Work through them sequentially.

Block A — Advanced Data Structures and Trees (6–8 weeks)

1. Segment tree with lazy propagation (Week 1–2). This is a major milestone. Implement range assign, range add, range min/max queries as separate exercises before combining.
2. Euler tour / DFS in-out flattening — flatten trees to arrays (Week 2)
3. Binary lifting and LCA via binary lifting (Week 3)
4. LCA via Euler tour + RMQ (Week 3)
5. Heavy-Light Decomposition + segment tree (Week 4–5). This is the hardest implementation in this block. Implement path sum, path max, and path update queries.
6. Centroid decomposition (Week 5–6)
7. DSU on tree / small-to-large on tree (Week 6)
8. Rerooting technique — re-rooted tree DP (Week 7)
9. Persistent segment tree (Week 7–8) — implement; template is acceptable for contests

Practice: CF problems tagged hld, centroid, tree, segment tree in 2100–2400 range.

Block B — Advanced Graphs (4–6 weeks)

1. Tarjan's SCC (Week 1)
2. Kosaraju's SCC (Week 1)
3. Bridges and articulation points (Week 2)
4. Biconnected components (Week 2)

5. 2-SAT — implication graph + SCC (Week 3). Solve at least 5 problems before moving on. The trick is modeling, not implementation.
6. Euler path and circuit — Hierholzer's algorithm (Week 3)
7. Bipartite matching — Hopcroft-Karp (Week 4)
8. Max flow — Dinic's algorithm (Week 4–5)
9. Block-cut tree and bridge tree (Week 5) — template level is sufficient
10. Min-cost max-flow (Week 6) — template level is sufficient
11. **[ADDED] Directed MST — Chu-Liu/Edmonds algorithm (Week 6). Template level is sufficient; focus on recognizing when a problem requires directed spanning tree vs undirected.**

Block C — Advanced DP (4–6 weeks)

1. Convex hull trick — monotone version (Week 1)
2. Convex hull trick — with binary search (Week 1–2)
3. Divide and conquer DP optimization (Week 2–3)
4. Knuth's optimization (Week 3)
5. Aliens trick / WQS binary search (Week 4). This is harder than it looks. Spend extra time. Template is acceptable.
6. Slope trick (Week 4–5)
7. **[ADDED] Markov chain DP — probability transitions, absorbing states, expected cost (Week 5). This extends expected value DP from Tier 3 into multi-state systems. Spend time on problems where the state space requires careful design.**
8. DP + Aho-Corasick automaton (Week 5–6)
9. Profile and broken profile DP (Week 6) — template level

Block D — Math, Strings, Geometry, Patterns (6–8 weeks)

1. FFT — polynomial multiplication (Week 1–2). Implement and understand convolution. Not just copy-paste.
2. NTT — FFT mod prime (Week 2)
3. XOR basis / linear basis over GF(2) (Week 2)
4. Gaussian elimination over reals and GF(2) (Week 3)
5. Aho-Corasick automaton (Week 3)
6. Suffix array + LCP array (Week 3–4). Implement it. Understand the LCP array deeply — most suffix array problems require LCP tricks, not just the array itself.
7. Suffix automaton (Week 4–5). Template level; but understand the state structure. DP on SAM requires knowing what states represent.
8. Lagrange interpolation, Sprague-Grundy, game theory (Week 5–6)
9. Geometry — convex hull, segment intersection, point in polygon, rotating calipers (Week 6–7)
10. Ordered statistics tree, implicit treap (Week 7)
11. Parallel binary search, rollback DSU, bitset DP optimization (Week 7–8)

Practice throughout Block D: CF 2200–2500 range. Virtual Div. 1 contests.

Done when: You solve CF 2200–2400 problems with reasonable consistency. You are no longer blocked by technique. Your bottleneck is observation and creativity, not implementation.

Phase 5 — Reaching and Holding 2400

No new content to learn at this stage. This is about execution quality.

What to do:

- Compete in every Codeforces round you can, rated.
- Upsolve every problem you could not solve in round, without exception.

- Identify your weakest problem type by reviewing your contest history. If you repeatedly fail on string problems, geometry, or a specific DP pattern — do a focused sprint of 15–20 problems in that type.
- Learn Tier 5 topics on demand: when you upsolve a problem and the intended solution uses a Tier 5 technique, learn that technique then. Do not preemptively grind Tier 5.

At this level, 40–50% of what separates you from higher-rated players is observation quality, not algorithm breadth.

Common Mistakes

- Learning without immediately solving problems. You can describe the algorithm but freeze on actual problems. Fix: solve at least 3 problems on every topic the same day you learn it.
- Only solving easy problems in a topic. You build false confidence. The technique is obvious when labeled; you can't apply it when unlabeled. Fix: after 3 easy problems, force yourself into problems where the technique isn't signaled.
- Skipping upsolving because "I know the technique now." You learn to recognize solutions; you don't learn to generate them under pressure. Fix: always implement, even if you fully understood the editorial.
- Grinding randomly without topic focus. You improve slowly and have persistent weak spots. Fix: at Tiers 1–3, 70% topic-focused practice, 30% mixed contests. Flip this at Tier 4.
- Moving tiers before the current one is solid. You constantly re-learn basics under pressure; shaky foundations compound at every tier. Fix: take the "Done when" benchmarks seriously. Being honest here saves months.
- Reading accepted code as a learning shortcut. You pattern-match a solution; you don't internalize the reasoning. Fix: read editorial for the key idea only. Implement yourself. If your implementation fails after multiple real attempts, then look at code for 5 minutes, close it, and implement from memory.
- Treating implementation as optional at higher tiers. At Tier 4, people start templating before they can implement. You cannot debug a template you don't understand. Fix: implement every Tier 4 core topic at least once from scratch before templating it.

Resources

Resource	Use for
CSES Problem Set	Structured topic-by-topic practice, Tiers 1–3
CP-algorithms.com	Algorithm reference and explanations
Codeforces EDU	Segment tree, DSU, suffix array with structured problems
AtCoder DP Contest	26 DP problems covering most patterns, Tiers 2–3
Codeforces Problem Filter	Filter by tag and rating for topic-focused practice
Codeforces Gym	Past ICPC regionals for mixed practice at Tier 3–4
USACO Guide	Structured roadmap, good for Tiers 2–4 with curated problem lists

End of document.